

מידול מילים ורצפים

Embeddings של מילים

עד עכשיו עסקנו במספרים (למעט נתונים קטגוריים, שגם אותם הפכנו למספרים). איך נייצג מילים?

- אפשרויות: sparse: • קידוד one-hot
- ווקטורי co-occurrence
- ייצוג embeddings: dense

הבעיה בייצוג sparse הוא שהוא מגדיל מאוד את מספר המימדים. למשל בקידוד one-hot נצרך אוצר מילים V וגודל הווקטור יהיה $|V|$. בעיה נוספת ב-one-hot היא שהיא מפספסת את הקשרים בין מילים קרובות - המודל יאלץ ללמוד את הקשרים האלה בעצמו. כדי לשפר את זה, נרצה להקטין את מספר המימדים - ממרחב בגודל עשרות אלפי מימדים למרחב בגודל של כמה מאות. נרצה שבמרחב החדש הזה המילה cat תהיה יותר קרובה ל-dog, למשל, מאשר ל-refrigerator.

איך יודעים מה המשמעות של מילה לא ידועה? לפי ההקשר. ניקח את האינטואיציה הזאת, ונייצר מטריצה - נספור לכל מילה כמה פעמים היא הופיעה ליד מילה אחרת (עד כדי מרחק של כמה מילים). במטריצה הזאת, ווקטור מייצג מילה (המטריצה סימטרית). מניחים שמילים כמו dog cat ייוצגו ע"י ווקטורים יחסית קרובים.

זה נקרא co-occurrence. מה הרווחנו לעומת one-hot? סמנטיקה. אבל לא פתרנו את בעיית הגודל. נצטרך לטפל גם בזה. אפשר לטפל בזה באמצעות SVD (לא נעשה כאן PCA, כי המטריצה היא sparse), אבל נרצה ללמוד ישירות את הייצוג dense. למשל - אלגוריתם Word2Vec.

הרעיון: ללמוד ייצוג שמאפשר לנחש את המילה החסרה במשפט. אם יש ייצוג למילה מסוימת אפשר לנחש את המילים לידו - או הפוך.

עבור כל חלון, מנסים לחזות לפי המילה במרכז את המילים מסביב.

מגדירים likelihood:

$$L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

ומנסים למזער את log-likelihood.

מגדירים לכל מילה $w \in V$ שני ווקטורים:

v_w ווקטור שמייצג את w כשהיא במרכז.

u_w ווקטור שמייצג את w כשהיא לא במרכז.

ואז, כדי לחשב את ההסתברות לראות את המילה o מסביב (outside) כאשר המילה c במרכז (center) היא

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

איך מאמנים את זה? יש לנו פרמטרים:
 d מספר המימדים שרוצים

$|V|$ גודל אוצר המילים

כל מילה מיוצגת ע"י שני ווקטורים (u_w, v_w) . לכן הגודל של θ הוא $2d|V|$. מאתחלים במספרים אקראיים (לא באפסים) ומשתמשים בSGD (לא בGD, כי יש הרבה יותר מדי) יש שני ייצוגים לכל מילה - אפשר לעשות ממוצע, או שאפשר להשתמש רק ב- v .

- בעיות: מילים בסוף/התחלה
- טוקנים מיוחדים (למשל סימני פיסוק)
- מילים מאוד נדירות

ויש גם בעיה כללית בword embedding שזה לא מתחשב בcontext. בשביל זה צריך Contextual Embeddings, שבו הווקטור מתחשב בכל המשפט מסביב למילה.

רצפים

במקום למדל מילים בודדות, אנחנו רוצים ללמוד למדל רצפים. Word2Vec לא עוזר לנו - הוא תופס את המשמעות של מילים, אבל לא של מילים בתוך רצף. הגישה הכי קרובה שראינו עד עכשיו היא CNN - אבל גם היא יכולה למדל רק חלון קטן מסביב.

רוצים: 1. לתמוך באורך משתנה.

2. להתחשב בסדר.

3. להתחשב בתלויות מאוד ארוכות אחורה.

נשים לב שאם אנחנו מצליחים למדל משמעות של מילה בתוך הקשר - זה אומר שהצלחנו למדל את כל הרצף.

נסיון שלא צלח - RNN (Recurrent Neural Networks)

הרעיון: לכל input x_t (שיכול להיות ייצוג של מילה ב-Word2Vec או כל ווקטור אחר (אם לא מדובר במילים)) והoutput כולל גם רכיב נסתר h_t שמצטרף למילה הבאה (זה recurrence)

RNN עצמו הוא לא deep, אבל אפשר להפוך אותו לdeep אם מוסיפים עוד שכבות. הלמידה כאן לא כל כך עובדת. גם אי אפשר למקבל אותו.

טרנספורמרים

הארכיטקטורה היא encoder-decoder:

- הencoder ממפה את הקלט של הsymbols לרצף של ייצוגים שאפשר לעבוד עליהם.
- הdecoder לוקח את הייצוג הזה ואת הטוקנים שהוא כבר ייצר, וחוזר את הטוקן הבא ברצף.

לפני זה עושים tokenization. למה?

- חלוקה לפי whitespace לא עובדת מספיק טוב.
- תמיד יכולים להיות מילים שלא ראינו באימון - אבל אנחנו עדיין צריכים לייצג אותן.
- אם יש מילה חדשה, רוצים למנף את זה שאנחנו יודעים לפחות חלקים ממנה.
- ייצוג של דברים כמו קוד או URLים.

הרעיון: במקום ללמוד לפי אותיות או לפי מילים שלמות, הולכים על משהו באמצע.

המאמר המקורי עושה byte-pair encoding (BPE) - מתחיל עם תווים בודדים ומחפש זוגות של דברים שהופיעו הרבה פעמים ביחד. אם רואים זוג כזה - מחברים, ומקבלים ככה טוקן חדש (שגם לו מחפשים זוגות)

זה מבטיח שאין דבר כזה OOV (out of vocabulary) - כי כל דבר אפשר להרכיב מטוקנים, גם אם - במקרה הכי גרוע - הם אותיות בודדות.

זה גם מקטין את $|V|$ ממאות אלפי מילים לכמה עשרות אלפי טוקנים.

במקום ללמוד embeddings של הטוקנים, נלמד אותו כחלק מהמודל עצמו - בתור השלב הראשון במודל.

מכיוון שהembeddings הזה לא נותן משמעות לסדר של המילים - נוסיף את הסדר הזה באופן מלאכותי. כיו כל מיני נסיונות:

- סתם לשים את המספר הסיידורים של המיקום.
- להוסיף עוד פרמטרים בשביל ללמוד את המשמעות של המיקום.
- מה שעשו בסוף - להוסיף ווקטור (כלומר לחבר אותו element-wise) של סינוסים

$$\text{שמייצג את המיקום היחסי.} \begin{pmatrix} \sin \frac{i}{10000^{2 \cdot \frac{1}{d}}} \\ \cos \frac{i}{10000^{2 \cdot \frac{1}{d}}} \\ \vdots \\ \sin \frac{i}{10000^{2 \cdot \frac{d/2}{d}}} \\ \cos \frac{i}{10000^{2 \cdot \frac{d/2}{d}}} \end{pmatrix} \text{וקוסינוסים}$$

אחרי embeddings של הטוקנים, מגיע שלב הattention. בשביל זה עושים soft lookup. אם נגדיר lookup רגיל בתור:

$$\text{Result} = \sum_{i=1}^n I(q == k_i) \cdot v_i$$

(ואז הוא מחזיר רק את ה v_i עבורו $q == k_i$). אבל מה אם אין k_i מתאים? נעשה soft max:

$$w_i = \frac{e^{1 \cdot k_i}}{\sum_j e^{q \cdot k_j}} \quad \text{Result} = \sum_{i=1}^n w_i \cdot v_i$$

ואז עושים attention בין כל טוקן לכל הטוקנים האחרים (כולל עצמו). מכפילים כל טוקן ב3 מטריצות:

- מטריצה אחת לייצג את ה q_i - מה אני מחפש

- מטריצה אחת לייצג את ה k_i - מי שמחפש, מה אני יודע לתת לו?

- מטריצה אחת לייצג את ה v_i - התוכן עצמו במקרה שיש התאמה

משתמשים באותן מטריצות לכל הטוקנים, ואז מחשבים לכל הקלט $X \in \mathbb{R}^{n \times d}$:

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

$$\text{Attention} = \text{softmax}(QK^T) V$$

אבל זה צריך להיות Multi-Headed Self-Attention. במקום ללמוד שלשה אחת, לומדים h מטריצות שונות כאלה שכל אחת תלמד משהו אחר. כדי שהתוצאה הסופית תהיה במימד שאנחנו רוצים, בוחרים $d_k = d_v = \frac{d_{\text{model}}}{h}$. ככה כל attention יכול להסתכל על מילים אחרות.

כדי לנרמל את variance מחלקים ב $\sqrt{d_k}$ (כאשר d_k הוא מספר המימדים בווקטורי הkey (וגם הvalue))

בשלב הזה יש לנו רצף של v ים. על זה שמים שכבה של Feed Forward - שתי מטריצות עם ReLU באמצע שהופכות את הייצוג הקונטקסטואלי לfeature space יותר מורכב:

$$FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

את כל הדבר הזה עושים N פעמים שמשרשרים אחד על גבי השני. זה הופך את זה לdeep.

לבסוף זה מגיעים לdecoder - שלוקח בתור הקלט את הטוקנים שכבר ייצרנו בעבר. יש בו:

- טוקן מיוחד Start of Sequence ($\langle SOS \rangle$) - מתחילים ממנו כדי להבין מה הטוקן הבא. אחרי זה יש שני טוקנים שמהם יש לנו את הטוקן הבא, וכן הלאה.

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-06-22

-
- Masked Multi-Head Attention - שמים $-\infty$ על כל מי שמקדימה כדי שלא נוכל להסתכל קדימה (כי זה output ולא רוצים לתת למודל לרמות). הAttentionים שונים כאשר לעומת הencoder - אבל מרחב הembeddings זהה.
 - Cross-Attention - לוקח גם את הפלט של הMasked Multi-Head Attention מהoutput וגם את הפלט של הMulti-Headed Attention (לאחר הFeed Forward) מהencoder.
 - אחרי זה עושים עוד Feed Forward וMulti-Header Attention על הכל, מוסיפים שכבה לינארית, ועושים softmax.

GPT

מעריך את הencoder - עושה רק decoder. זה אומר שאין cross-attention.